

PROJECT NOTES

SkyGuard AI

Intelligent Real-Time Anomaly Detection for Automatic Weather Stations
— AI/ML Development Log —

Smart India Hackathon • AI/ML Core Track

Sanchal Kumar • Sagar Institute of Research & Technology, Bhopal

Covers: Project Foundation → Dataset Design → Anomaly Simulation → Physics Rule Engine

1. What I Am Building

SkyGuard AI is an intelligent anomaly detection system for Automatic Weather Stations (AWS). The core problem is simple to state but not simple to solve: an AWS sensor network sends in readings for temperature, pressure and humidity every day, and some of those readings are wrong. The system I'm building has to figure out, for every single reading, whether it is **normal**, a **sensor fault**, a **data transmission glitch**, or an actual **genuine weather event** that just happens to look unusual.

This last distinction is the whole point of the project. If a station suddenly reports 46°C with a strange pressure reading, a naive system would just call it "extreme weather." My system has to be smarter than that — it has to check whether nearby stations show the same pattern before deciding whether this is a broken sensor or a real heatwave.

Division of Work

The larger SkyGuard AI system has a full architecture — data ingestion over MQTT, a Java/Spring Boot backend, a TimescaleDB time-series database, and a React dashboard. Shreyash and I are responsible for only one part of this: the **AI/ML Anomaly Detection Core**. The rest is being handled by our teammates on backend and frontend. Our deliverable is a clean, well-tested ML pipeline that the backend can call through a simple API.

Our scope: Physics Rule Engine → LSTM Autoencoder (temporal patterns) → Spatial Correlation Engine → Isolation Forest (multivariate outliers) → Fusion Layer (final classification + confidence score) → Explainability (SHAP).

The Official Problem Statement

The SIH problem statement narrows the input down to exactly three sensor parameters: **Temperature (°C)**, **Atmospheric Pressure (hPa)**, and **Relative Humidity (%)**. It also explicitly asks for real-time detection, explainable reasoning behind each alert, confidence scores, and a system that scales across a large station network — not just a one-off script.

2. Deciding What the Dataset Should Look Like

I started with a base AWS dataset covering 14 stations across India, with daily readings from 2021 to early 2025 — about 20,888 rows in total, 15 columns: station identity, temperature (avg/min/max), humidity, wind speed, pressure, rainfall, elevation, and coordinates.

Keeping 10 Features, Training on 3

Since the official problem statement only asks for temperature, pressure and humidity, I had a choice: strip the dataset down to just those three, or keep everything. I decided to **keep all 10 numeric features** in the dataset, but restrict the actual anomaly injection and model training to just the three core ones. My reasoning: the extra features (elevation, coordinates, wind, rainfall) aren't inputs to the anomaly model, but they're essential context — coordinates are what the Spatial Correlation Engine will need to find "nearby" stations, and elevation explains why two stations can have very different normal pressure baselines.

Why this matters: it means the 15-column dataset I hand off to backend/frontend never has to change, while my ML side has everything it needs for every future step — I won't have to go back and regenerate the dataset again when I reach the Spatial Engine.

Choosing a Small, Focused Set of Anomaly Types

My first instinct was to model every anomaly type I could think of — sudden spikes, drops, drift, bias shift, cross-sensor inconsistency, duplicates, nulls, regional rain events, and more. I later decided against this. A large, overlapping anomaly taxonomy makes the model harder to evaluate and doesn't add much real value, since several types behave almost identically to a model. I narrowed it down to six anomaly types that are genuinely common and genuinely distinct:

Category	Type	What it represents
SENSOR_FAULT	SUDDEN_SPIKE	One-day abrupt jump/drop — classic sensor glitch
	STUCK_AT	Sensor freezes and repeats the same value for several days
	GRADUAL_DRIFT	Slow calibration decay over 6–12 days
TRANSMISSION_GLITCH	MISSING_RECORD	Communication failure — no data arrives at all
GENUINE_EVENT	REGIONAL_HEAT_EVENT	A real heatwave — whole region moves together
	REGIONAL_PRESSURE_DROP	A real low-pressure weather system passing through

3. Building the Anomaly Injection Pipeline

Since the base AWS data has no ground-truth labels for "this was a sensor fault," I built a synthetic anomaly injection pipeline — deliberately corrupting a small, controlled fraction of the clean data so I'd have labelled examples to train and evaluate against.

Code Structure

I organized this as separate, focused modules instead of one large script, matching my project's folder layout:

```
01_ml/03_src/02_anomaly/
├─ sensor_faults.py      ─ Sudden_Spike, Stuck_At, Gradual_Drift
├─ transmission_faults.py ─ Missing_Record
├─ genuine_events.py     ─ Regional_Heat_Event, Regional_Pressure_Drop
└─ injection.py         ─ orchestrator: loads data, runs every injector, saves output
```

Result

Running this pipeline against the base data gave me a final labelled dataset of 20,832 rows (a few rows are deliberately removed to simulate missing transmissions):

Category	Rows	Share
NORMAL	19,654	94.3%
SENSOR_FAULT	816	3.9%
GENUINE_EVENT	362	1.7%

4. Quality-Checking My Own Dataset

Before trusting this dataset for model training, I ran it through several rounds of validation. This turned out to be important — I found and fixed real bugs each round, and I'm documenting them honestly here rather than pretending the first version was perfect.

Round 1 — Physics Consistency & Realistic Ranges

What I found: My regional heat events were raising `avg_temp` without touching `min_temp` / `max_temp` — so a genuine heatwave could end up with an average temperature higher than its own recorded maximum, which is physically impossible. I also found that my sudden-spike injections could push pressure to 1601 hPa and humidity to 125% — values that don't exist in the real world.

How I fixed it: I scaled `min_temp` and `max_temp` by the same factor as `avg_temp` during a heat event, so the $\text{min} \leq \text{avg} \leq \text{max}$ relationship stays intact. I also added realistic clip bounds per feature (e.g. pressure clipped to 940-1050 hPa) so an injected fault is "wrong but plausible," not physically absurd.

Round 2 — Rolling Statistics Were Stale

What I found: I had computed 7-day rolling mean/std columns *before* injecting anomalies, so those columns reflected clean, pre-injection values — not what a live pipeline would actually observe once the data stream is corrupted.

How I fixed it: I moved the rolling-stats computation to *after* all injections, so those columns now genuinely reflect the "as-observed" corrupted stream.

Round 3 — An Index Corruption Bug

What I found: This was the most serious issue. My missing-record function dropped rows and then called `reset_index()` — which silently reshuffled every row's position. Later injectors (the regional events) were still using row positions recorded before that reset, so they ended up writing into the *wrong* rows — overwriting a couple of already-labelled sensor-fault events with genuine-event data.

How I fixed it: I stopped resetting the index mid-pipeline, and only reset it once, right at the very end, after every injector has finished. I re-validated event-to-row mapping across the whole dataset afterward and confirmed zero corrupted rows.

Round 4 — Making Sure My Numbers Are Real, Not Just Convenient

While building the Physics Rule Engine (next section), I noticed my regional pressure-drop events were using a magnitude of -30 to -60 hPa. That's larger than the pressure network's entire normal range, which was suspicious. Before just shrinking the number to make my physics checks behave better, I stopped and asked whether that would be scientifically justified or just convenient — because tuning synthetic data purely to satisfy a detector is a form of data fabrication, not a bug fix.

I looked up real Indian meteorological data on monsoon depressions and western disturbances — the actual weather systems responsible for regional pressure drops over India. The data showed that even a strong monsoon depression only drops surface pressure by about 2-8 hPa over a region, not tens of hPa. I recalibrated my injection to -3 to -8 hPa, grounded directly in that real-world figure — and documented the reasoning in the code itself, so the justification is visible to anyone who reads it later.

5. Building the Physics Rule Engine

This is the first layer of my actual detection system — a deterministic, fast, rule-based checker that needs no training at all. It runs first in the pipeline and is meant to catch the "obviously wrong" cases cheaply, so the heavier ML layers only need to handle the subtler patterns.

Three Kinds of Checks

- **Range check** — is the value within physically possible bounds? I used two tiers: a hard limit (physically impossible) and a softer limit (unusual but not impossible, e.g. an extreme heatwave).
- **Consistency check** — does `min_temp ≤ avg_temp ≤ max_temp` hold for that day?
- **Step-change check** — did the value jump further than physically realistic compared to the previous day at the same station?

Why the step-change check matters: I discovered that most of my temperature and humidity spikes actually land *inside* the normal observed range even after the spike — 51 out of 60 temperature spikes, for example. A simple range check alone would completely miss these. Only by checking how much the value *jumped* in a single day could I catch them.

Standalone Performance (Before Any ML Layer)

Anomaly Type	Caught by Physics Alone	Notes
SUDDEN_SPIKE	70.7%	Good — mostly caught by the step-change check
GRADUAL_DRIFT	63.8%	Good — many drift days eventually cross a bound
STUCK_AT	3.0%	Expected — this needs the LSTM to see the pattern
REGIONAL_HEAT_EVENT	0.5%	Correct — a genuine event shouldn't look like a violation
REGIONAL_PRESSURE_DROP	0.6%	Correct — after the magnitude recalibration above

False positive rate on NORMAL rows: **0.77%**. False-flag rate on GENUINE_EVENT rows overall: **0.55%** — down from 41% before I fixed the magnitude calibration issue.

This result is exactly what I want at this stage: the Physics Engine is strong on the anomalies that are meant to be "obviously wrong," and it deliberately stays quiet on genuine events and subtle multi-day patterns — because those are the jobs of the Spatial Correlation Engine and the LSTM, which come next.

6. Where I Am Right Now

Phase	Status
Problem definition & scope	DONE
Dataset design (10 features, 3 model inputs)	DONE
Synthetic anomaly injection pipeline	DONE
Dataset validation (4 rounds, bugs fixed)	DONE
Physics Rule Engine	DONE
LSTM Autoencoder (temporal patterns)	NEXT
Spatial Correlation Engine	NEXT
Isolation Forest (multivariate outliers)	NEXT
Fusion Layer + Confidence Scoring	NEXT
Explainability (SHAP)	NEXT

One Thing I'm Tracking for Later

My rolling-stat columns currently use `min_periods=1` , which creates a narrow blind spot: if an anomaly's row is preceded by a missing value (or is right at the start of a station's history), the rolling mean ends up equal to the anomalous value itself, making deviation-based detection blind to it for that one row. This only affects the Isolation Forest step later on — Physics, LSTM and Spatial Correlation all read raw values and are unaffected. I've deliberately postponed this fix until I actually reach the Isolation Forest step.

What Comes Next

My next milestone is the LSTM Autoencoder — instead of judging one reading at a time, it will look at a 7-day rolling window per station and learn what a "normal" temporal pattern looks like, which is what I need to catch stuck sensors and gradual drift reliably. After that, the Spatial Correlation Engine will let me compare a station's reading against its geographic neighbors — the key piece that actually tells a sensor fault apart from a real regional weather event.